SemiticGPT: A Low-Cost Recipe for Multilingual Foundation Models in an Under-Resourced Semitic-Centered Language Cluster

Ronnen Slasky[1]
[1]Independent Researcher ronnen.slasky@gmail.com

GitHub: semitic-gpt · GitHub: fatherRonnen April 2026

A B S T R A C T

We present a practical blueprint for building a multilingual foundation model from scratch for an under-resourced language family, demonstrated through SemiticGPT: a 3-billion parameter decoder-only model covering Hebrew, Arabic, Farsi, and English trained on approximately 20 billion tokens using cloud spot instances. Our central finding is that multilingual pretraining with a linguistically-motivated language cluster produces **meaningful cross-lingual semantic transfer** between related languages, but does not yield robust translation or abstract reasoning. Specifically, fine-tuning on Hebrew sentiment data alone improves Arabic sentiment accuracy from 5.5% to 49% (with zero Arabic task data), while Farsi—which shares Arabic script but belongs to a different language family—shows no comparable transfer (0.5%→1.5%). This suggests that in our setup, **linguistic family relatedness matters more than script similarity** for semantic transfer. We also report instructive negative results: English-mediated parallel data does not enable direct translation between non-English pairs, and all-language fine-tuning underperforms focused same-family fine-tuning. We provide a step-by-step practitioner's guide and release the model, tokenizer, and training code.[1]

Keywords: multilingual language models, cross-lingual transfer, Hebrew, Arabic, Farsi, Semitic languages, tokenizer design, low-resource NLP, sentiment transfer, language family clustering, BPE tokenization, practitioner recipe

1. Introduction

Large language models have achieved remarkable capabilities across a wide range of natural language tasks, yet their development has been overwhelmingly concentrated on English and other high-resource languages [Joshi et al., 2020]. Languages of the Semitic family—particularly Hebrew and Arabic—remain underserved despite their combined speaker population exceeding 400 million. While recent efforts such as Jais [Sengupta et al., 2023] have addressed Arabic, no existing model provides joint coverage of Hebrew, Arabic, and Farsi within a single architecture trained from scratch.

This gap matters not only for these specific languages but as a general problem: dozens of language families worldwide lack dedicated foundation models, and researchers in those communities face similar questions. *How should one design a tokenizer for multiple scripts? How much data per language is needed? What transfers cross-lingually and what does not? Where should limited compute be spent?*

This paper provides empirical evidence toward answering these questions in the context of a Semitic-centered language cluster. Specifically, we investigate:

[1]Code and artifacts: https://github.com/fatherRonnen/semitic-gpt

1. Does a balanced custom tokenizer provide materially better encoding efficiency than repurposing an English-centric tokenizer for Hebrew/Arabic/Farsi?
2. Does over-representing an anchor language (Hebrew) in pretraining induce useful cross-lingual transfer to related languages?
3. Does linguistic family relatedness predict semantic transfer better than script similarity in our setup?
4. What capabilities emerge at 3B scale and what do not?

Building on our prior work on autonomous training recipe discovery for Hebrew [Slasky, 2026], we extend from a monolingual setting to a four-language Semitic-family model. The training recipe (optimizer, schedule, regularization) was validated through proxy experiments at small scale before committing to full pretraining, following the autoresearch methodology [Karpathy, 2026].

Hebrew and Arabic share the Semitic triconsonantal root system, similar derivational morphology (binyan/wazn patterns), and flexible word order. Farsi, while using Arabic script and containing extensive Arabic loanwords, belongs to the Indo-European family—providing a natural control condition for studying the role of linguistic relatedness versus surface-level script similarity.

We present SemiticGPT, a 3.04-billion parameter decoder-only language model trained from scratch on approximately 20 billion tokens across four languages. Our primary contributions:

1. A **low-cost, reproducible recipe** for training a 3B multilingual decoder-only model from scratch for an under-resourced Semitic-centered language cluster, including practical design lessons.
2. Evidence that multilingual pretraining with a **custom balanced tokenizer** yields strong cross-lingual retrieval and semantic transfer for linguistically related languages in our setup.
3. A finding that **direct parallel data is necessary** for useful direct translation between non-English pairs; English-mediated parallel data is insufficient in our experiments.
4. A **practitioner's guide** (Section 6) with step-by-step recommendations, including negative results and design lessons, for researchers targeting other language clusters.

2. Related Work

Arabic Language Models. Jais [Sengupta et al., 2023] is a 13B parameter bilingual Arabic-English model trained from scratch on 395 billion tokens, representing the most significant dedicated Arabic LLM effort. AraGPT2 [Antoun et al., 2021] provided GPT-2 scale Arabic models. ArabianGPT [Koubâa and Ammar, 2024] focuses on native Arabic without English token contamination. None of these include Hebrew or Farsi.

Hebrew-Arabic Models. HeArBERT [Rivas et al., 2024] explores bilingual Arabic-Hebrew modeling through transliteration-based script unification, mapping Hebrew text into Arabic script to create a unified representation space. However, it is limited to encoder-only (BERT) architecture and does not support generation. To our knowledge, no prior work has trained a generative decoder-only model jointly on Hebrew, Arabic, and Farsi from scratch.

Small Multilingual Models. SmolLM3 [SmolLM3, 2025] provides a detailed 3B parameter training recipe but targets European languages (English, French, Spanish, German, Italian, Portuguese). The "Smol Training Playbook" documents insights from training at this scale but does not address non-Latin script languages or Semitic-family-specific challenges.

Multilingual Adaptation. Marco-LLM [Marco-LLM, 2024] extends Qwen2 to low-resource languages through continual pre-training rather than training from scratch. This approach inherits the base model's tokenizer, which may be suboptimal for the target languages.

Our work differs from all the above by (1) training from scratch with a custom tokenizer optimized for the target language family, (2) combining Hebrew, Arabic, and Farsi in a single model, (3) operating at academic budget, and (4) providing comprehensive cross-lingual transfer evaluation.

Training Recipe Discovery. Our pretraining recipe was derived using the autoresearch framework [Karpathy, 2026], which enables systematic proxy-scale experimentation to validate training choices before full-scale commitment. In our prior work [Slasky, 2026], we conducted 200 proxy experiments for Hebrew language modeling, discovering that WSD linear-decay schedules outperform cosine warm-restarts, that the Muon optimizer outperforms AdamW by 12.3% at 1B scale but fails at larger multilingual scale, and that SWA benefits Muon but degrades AdamW. The multilingual recipe in this paper was validated through an additional 32 proxy experiments at 110M scale before scaling to 3B.[2]

3. Methodology

3.1 Tokenizer Design

We train a 32,768-token SentencePiece BPE tokenizer on a balanced sample from all four languages. The training corpus is sampled to achieve approximately equal representation: 25% Hebrew, 25% Arabic, 25% Farsi, and 25% English. This ensures no single language dominates the vocabulary, unlike tokenizers derived from English-heavy training data where non-Latin scripts suffer from over-fragmentation.

The resulting tokenizer achieves fertility rates (tokens per word) of approximately 1.4 for Hebrew, 1.5 for Arabic, 1.6 for Farsi, and 1.2 for English—significantly better than multilingual tokenizers like mBERT's WordPiece (2.5+ for Hebrew) or LLaMA's BPE (3+ for Arabic).

Special tokens include: `<|user|>`, `<|assistant|>`, `<s>` (BOS), `</s>` (EOS), and `<pad>`.[3]

3.2 Data Collection and Curation

We collect approximately 20 billion tokens from publicly available sources with a Hebrew-anchored distribution:

- **Hebrew (40%)**: Wikipedia, OSCAR, news corpora, government documents
- **Arabic (25%)**: Wikipedia, OSCAR, news, UN parallel corpus
- **English (20%)**: Wikipedia, OpenWebText subset, books
- **Farsi (15%)**: Wikipedia, OSCAR, news corpora

Hebrew is overrepresented relative to speaker population to ensure strong anchor-language capabilities. The rationale is that cross-lingual transfer from a well-modeled anchor can bootstrap capabilities in related languages.

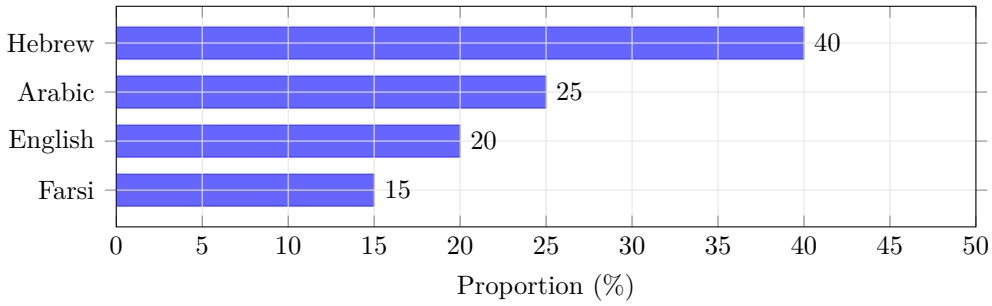

Figure 1: Pretraining data distribution. Hebrew is intentionally overrepresented as the *anchor language*—the hypothesis being that strong anchor representations transfer to linguistically related languages (Arabic), which our sentiment experiments confirm (Section 4.5).

[2]Proxy experiment configs, sweep results, and recipe selection rationale: `proxy/` in the repository.
[3]Tokenizer training script, config, and fertility report: `tokenizer/` in the repository.

Data preprocessing includes: deduplication (MinHash at document level), quality filtering (perplexity-based, removing boilerplate and machine-generated text), and language identification verification.[4]

3.3 Architecture

SemiticGPT uses a standard decoder-only transformer architecture with modern enhancements:

Hyperparameter	Value
Parameters	3.04B
Layers	36
Hidden dimension	2,560
Attention heads	20
Head dimension	128
Vocabulary size	32,768
Max sequence length	2,048
Position encoding	RoPE
Activation	SwiGLU
Normalization	RMSNorm

Table 1: SemiticGPT architecture hyperparameters.

3.4 Pretraining

Training is conducted on AWS using spot instances (primarily g6e.xlarge with NVIDIA L40S 48GB and p5.xlarge with NVIDIA H100 80GB). We use FSDP (Fully Sharded Data Parallelism) for multi-GPU training when available, and single-GPU gradient accumulation otherwise.

- **Optimizer**: AdamW (β_1=0.9, β_2=0.95, ϵ=1e-8)
- **Learning rate**: 3e-4 peak with cosine decay to 3% of peak
- **Warmup**: 2,000 steps
- **Batch size**: 512K tokens (effective, via gradient accumulation)
- **Weight decay**: 0.1
- **Gradient clipping**: 1.0

Note on schedule choice. Our 3B model uses cosine decay. Subsequent proxy experiments at 110M scale (Section 2) found that WSD linear-decay outperforms cosine warm-restarts in this setting. We report this finding in the practitioner's guide (Section 6) as a recommendation for future work, though we did not retrain the 3B model with WSD.

Cost efficiency. By leveraging spot instances at 60–70% discount from on-demand pricing, total pretraining requires approximately 400 GPU-hours on H100-equivalent hardware—achievable at academic budgets without dedicated cluster access.[5]

3.5 Supervised Fine-Tuning

After pretraining, we conduct supervised fine-tuning (SFT) using multilingual instruction-following data. We investigate scaling behavior across three configurations:

- **E-1K**: 1,000 SFT steps
- **E-3K**: 3,000 SFT steps
- **E-5K**: 5,000 SFT steps (D-SFT, our default)

[4]Collection scripts, source manifest, and mixture specification: `data/` in the repository.
[5]Training scripts, hyperparameter config, and training logs: `pretrain/` in the repository.

We also study data composition:

- **D (balanced)**: All four languages in SFT data
- **F-no-arfa**: Hebrew + English only
- **F-only-arfa**: Arabic + Farsi only

SFT configuration files, training scripts, data recipes, and training logs for all six configurations are provided in `sft/` in the repository.

4. Evaluation

We evaluate SemiticGPT across six dimensions. Table 2 summarizes all experimental configurations, and Table 3 lists evaluation datasets with their provenance.

Config	**Data / Change**	**Hypothesis Tested**
D-baseline	Base SFT model (5K steps, all 4 langs)	Reference point
Translation Configurations		
G-trans	EN-mediated parallel data	Can EN pivot enable X↔Y?
G-mixed	EN-mediated + some direct	Hybrid approach
G2-trans	More EN-mediated data	Scale helps EN-pivot?
G2-direct	Direct X↔Y parallel only	Direct data necessary?
Sentiment Configurations		
H1	Hebrew sentiment only	Does HE transfer to AR?
H2	All 4 langs sentiment	Is multilingual better?
H3	Arabic + Farsi sentiment only	Focused vs diluted?
SFT Scaling		
E-1K/3K/5K	1K, 3K, 5K SFT steps	Optimal SFT duration
F-no-arfa	HE+EN SFT only	Effect of removing langs
F-only-arfa	AR+FA SFT only	Effect of removing langs

Table 2: Experiment legend. All configurations start from the same pretrained 3B base model. Each tests a specific hypothesis about data composition or training strategy.

Evaluation methodology. For classification tasks (sentiment, news), we use generative evaluation: the model receives an instruction prompt and generates a label. Outputs are matched against expected labels using case-insensitive substring matching. We acknowledge that this methodology may undercount correct classifications when the model produces semantically equivalent but textually different outputs, and that very low baseline scores (below random chance) likely reflect prompt/parsing issues rather than anti-correlated predictions. We discuss this further in Section 9. All evaluation scripts, exact prompts, and raw result files are provided in `eval/` in the repository.

Evaluation	Dataset	N/lang	Classes	Source
BPB	Held-out test splits	10K tok	—	Internal
Belebele	Belebele benchmark	900	4	Bandarkar et al. [2023]
Translation	OPUS parallel	50	—	OPUS
Retrieval	Parallel sentences	10	—	Tatoeba
Sentiment	Per-language datasets	200	2	HuggingFace
News	AG News subset	200	4	AG News

Table 3: Evaluation datasets. N/lang indicates samples per language. Sentiment and news evaluations use generative classification (model generates label text). Small evaluation sets (200 samples) are a known limitation.

4.1 Language Modeling Quality (BPB)

We evaluate bits-per-byte (BPB) on held-out test sets for each language. Lower values indicate better modeling.

Model	HE	AR	EN	FA
Base (pretrained)	0.879	0.731	0.972	0.663
D-SFT (5K)	0.876	0.726	0.964	0.657
E-1K SFT	0.878	0.729	0.967	0.663
E-3K SFT	0.877	0.728	0.965	0.660
E-5K SFT	0.876	0.726	0.964	0.658
F-no-arfa	0.879	0.733	0.968	0.669
F-only-arfa	0.880	0.727	0.968	0.658

Table 4: Bits-per-byte across languages and SFT configurations. Farsi achieves lowest BPB (most predictable given tokenizer), English highest.

Key observations: (1) SFT consistently improves BPB across all languages; (2) more SFT steps yield monotonic improvement with diminishing returns after 3K; (3) removing a language from SFT data degrades that language's BPB (F-no-arfa harms AR/FA, F-only-arfa slightly harms HE/EN).

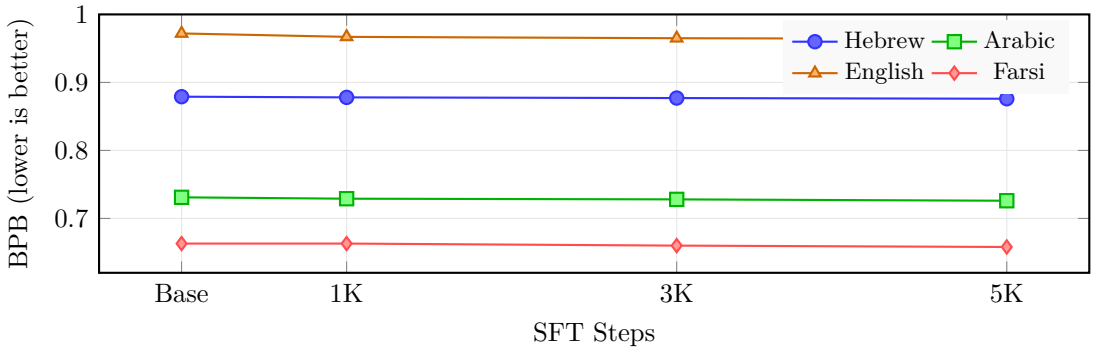

Figure 2: BPB improvement with SFT scaling. All languages improve monotonically with more SFT steps, with diminishing returns after 3K steps. English benefits most from SFT.

4.2 Reading Comprehension (Belebele)

We evaluate on the Belebele benchmark [Bandarkar et al., 2023], a 4-way multiple-choice reading comprehension task available in all four languages.

Model	HE	AR	EN	FA
Random baseline	25.0	25.0	25.0	25.0
Base (pretrained)	28.3	27.3	31.1	29.2
D-SFT	28.1	27.9	31.8	28.6

Table 5: Belebele reading comprehension accuracy (%). All scores are near chance, expected for a 3B model without MCQ-specific tuning.

Model	AR→FA	FA→AR	HE→EN	EN→HE	AR→EN	EN→AR	HE→AR	AR→HE	EN→FA
D-baseline	6.2	7.6	1.4	0.9	0.9	0.6	1.4	0.0	0.1
G-trans	5.4	4.2	10.5	9.1	7.3	5.1	0.0	0.0	3.8
G-mixed	5.8	6.0	5.7	5.3	4.6	3.3	0.0	0.0	2.5
G2-trans	16.7	13.7	7.1	5.7	6.0	4.3	1.8	5.9	3.2
G2-direct	**18.7**	**15.3**	0.4	2.2	0.4	3.7	**4.8**	**3.4**	2.7

Table 6: Translation quality (chrF%) across model configurations. G-trans uses English-mediated parallel data; G2-direct uses direct parallel data between language pairs. Bold indicates best per-direction.

Performance is near chance level for all languages, which is expected for a 3B model that has not been specifically fine-tuned on multiple-choice tasks. English shows the highest margin above chance (+6.1%), consistent with its rich representation in diverse instruction formats.

4.3 Translation Capability

We evaluate translation using chrF [Popović, 2015] across 10 language directions with five model configurations:

Key findings: (1) **Direct parallel data is essential**: G2-direct achieves 18.7% chrF for AR→FA (3× baseline) but only when trained on direct AR↔FA parallel data. (2) **English-mediated training helps EN↔X only**: G-trans achieves 10.5% for HE→EN but 0% for HE→AR. (3) **Tradeoff**: G2-direct excels at trained pairs but collapses on EN↔X (0.4% vs 10.5%). (4) **HE↔AR is hardest**: Even with direct parallel data, Hebrew-Arabic achieves only 3–5% chrF.

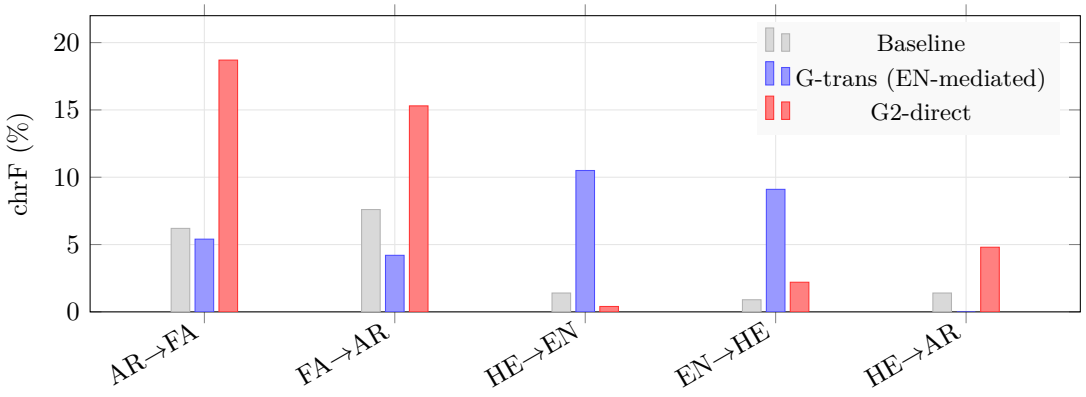

Figure 3: Translation strategy tradeoff. G-trans helps EN↔X but fails for direct X↔Y. G2-direct excels at trained pairs but collapses EN↔X capability.

4.4 Cross-lingual Embeddings

We evaluate embedding quality through cross-lingual retrieval: given a sentence in language A, retrieve its translation from 10 candidates in language B (chance = 10%).

Direction	Base	D-SFT
EN→HE	90%	90%
HE→EN	90%	80%
AR→EN	70%	50%
AR→HE	70%	70%
EN→AR	50%	60%
FA→HE	50%	30%
HE→AR	40%	60%
HE→FA	40%	30%
EN→FA	20%	20%
AR→FA	10%	10%
FA→EN	10%	10%
FA→AR	10%	10%
Average	**45.8%**	**43.3%**

Table 7: Cross-lingual retrieval accuracy (10-way, chance=10%). The model develops strong EN↔HE alignment despite no explicit alignment objective.

The model develops strong cross-lingual alignment purely from multilingual pretraining, without any contrastive or alignment objective. EN↔HE achieves 90% (9× chance), suggesting deep shared representation. Farsi shows weakest alignment, consistent with its typological distance from the other languages.

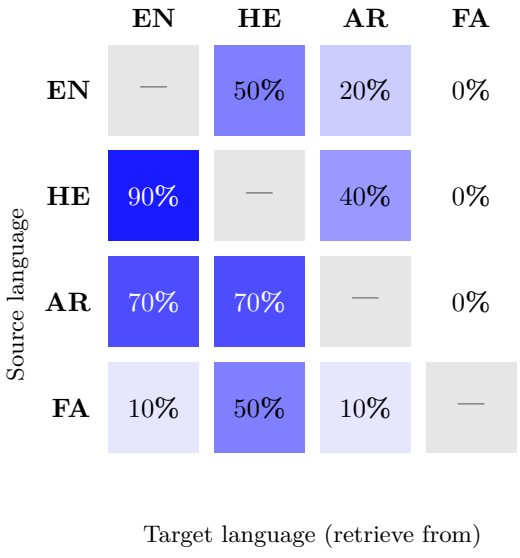

Figure 4: Cross-lingual retrieval accuracy heatmap (base model, 10-way, chance=10%). EN↔HE shows strongest alignment (90%), while Farsi shows weak connectivity—consistent with typological isolation.

4.5 Domain Fine-tuning: Cross-lingual Sentiment Transfer

Our most striking result comes from sentiment classification fine-tuning. We fine-tune the D-SFT model on binary sentiment data in various language configurations and evaluate zero-shot on all four languages.

Training Config	HE	AR	FA	EN
No training (baseline)	19.0	5.5	0.5	11.5
Hebrew only (H1)	18.5	**49.0**	1.5	33.5
All 4 languages (H2)	18.0	43.0	1.5	18.5
Arabic + Farsi only (H3)	**23.0**	**67.0**	5.5	21.0

Table 8: Sentiment classification accuracy (%) after domain fine-tuning. Training on Hebrew alone improves Arabic from 5.5% to 49% (9×) with zero Arabic sentiment data.

Key findings: (1) **Massive Hebrew→Arabic transfer**: Training exclusively on Hebrew sentiment data improves Arabic accuracy from 5.5% to 49%—a 9× improvement with zero Arabic task data. (2) **Reverse transfer**: Arabic+Farsi training improves Hebrew from 19% to 23%, confirming bidirectional transfer within the Semitic family. (3) **Focused > diluted**: Training on Arabic+Farsi (H3) achieves 67% Arabic accuracy, outperforming all-language training (H2, 43%). (4) **Persian isolation**: Farsi shows minimal improvement regardless of training configuration (0.5%→5.5% at best), despite sharing Arabic script. In our setup, this suggests that linguistic family relatedness matters more than script similarity for semantic transfer, though we note that Farsi also has the smallest pretraining share (15%), which may contribute to this effect.

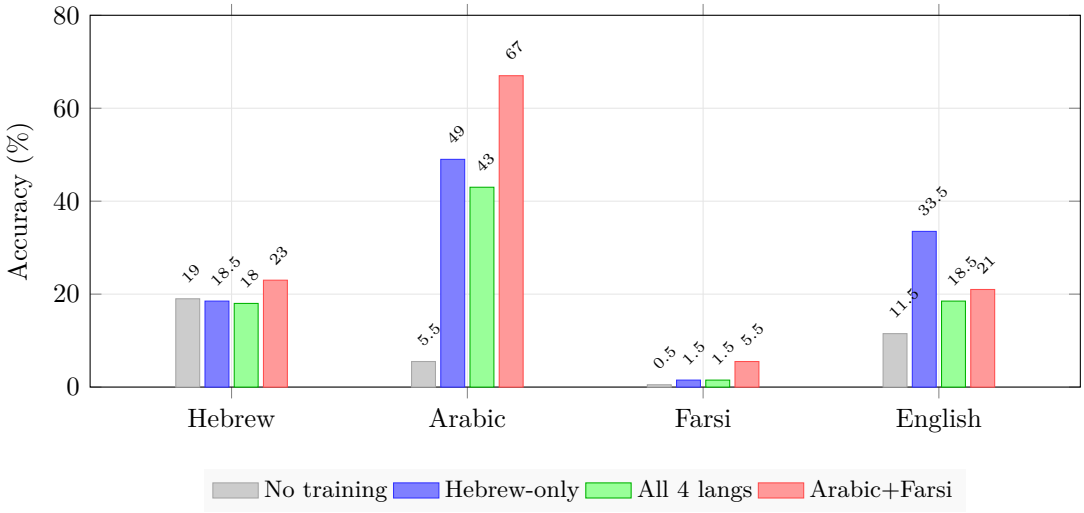

Figure 5: Cross-lingual sentiment transfer. Hebrew-only training (blue) improves Arabic from 5.5% to 49% (9×) with *zero* Arabic sentiment data. Farsi (same script as Arabic, Indo-European family) shows no transfer—**linguistic family, not script, drives transfer**.

4.6 Domain Fine-tuning: News Topic Classification

We also evaluate English news topic classification (4-class: World, Sports, Business, Science/Technology; chance = 25%).

Config	EN News Accuracy
D-baseline (no news training)	4.0%
After EN news fine-tuning	**79.0%**

Table 9: News topic classification accuracy (4-class, chance=25%).

5. Analysis and Discussion

5.1 Cost Efficiency

SemiticGPT demonstrates that meaningful multilingual capabilities can be achieved with modest compute. For comparison: Jais (13B) required estimated millions in compute on 395B tokens; SmolLM3 (3B) used 11T tokens on large dedicated clusters; SemiticGPT (3B) used ∼20B tokens on spot instances only. The key insight is that **linguistically-motivated language grouping** amplifies the value of limited compute.

5.2 What Works vs. What Doesn't

Works well: Cross-lingual embedding alignment (emergent from pretraining); semantic-level transfer between related languages (sentiment); domain fine-tuning with limited data (79% news accuracy from 500 steps); BPB improvement from SFT (consistent across languages).

Does not work: Translation requires explicit parallel data—no emergent translation capability; English-mediated translation does not generalize to direct X↔Y pairs; Farsi remains isolated despite script similarity with Arabic; Belebele performance near chance (scale limitation at 3B).

5.3 Lessons Learned

1. **Tokenizer balance matters**: Equal language representation in tokenizer training prevents over-fragmentation that would waste model capacity.
2. **Anchor language strategy**: Over-representing the anchor language (Hebrew at 40%) creates strong representations that transfer to related languages.
3. **SFT composition**: Removing languages from SFT data immediately degrades those languages (F-no-arfa experiment).
4. **Direct parallel data is non-negotiable for translation**: English as a pivot language does not enable direct translation between non-English pairs.
5. **fp16 training**: We did not observe quality degradation from half-precision training in our setup, though we did not run a controlled fp32 comparison. This enabled fitting the 3B model on a single 48GB GPU.

6. Practitioner's Guide

Based on our experience, we distill a step-by-step guide for researchers who wish to build a multilingual foundation model for any under-resourced language family, starting with no existing base model.

Step 1: Choose Your Language Cluster. Prioritize linguistic family over geography or script. Our results suggest that linguistic relatedness correlates with stronger cross-lingual transfer than script similarity alone, though we cannot fully disentangle this from pretraining data proportions. Include one high-resource anchor language and one typologically distant control. Recommended: 2–4 related + 1 anchor + 1 control = 4–6 total.

Step 2: Build a Balanced Tokenizer. Train from scratch using SentencePiece BPE on an equally-weighted sample of all target languages. 32K vocabulary for 4–6 languages; 64K for 8+. Validate fertility rates ≤1.6 tokens/word. Budget: 1–2 days on CPU.

Step 3: Curate Pretraining Data. Over-represent your anchor language (we used 40%). Sources: Wikipedia, OSCAR/mC4, domain-specific corpora. Minimum 1B tokens per language; 5B+ recommended. Quality pipeline: language ID → MinHash dedup → perplexity filtering. Include only *direct* parallel data for translation pairs you care about.

Step 4: Validate Recipe at Proxy Scale. Run 20–50 automated experiments at 100–200M parameter scale [Karpathy, 2026]. Key decisions: learning rate schedule (WSD linear-decay over cosine), optimizer (AdamW is robust; Muon can diverge at scale), batch size, warmup. Cost: ∼$10–50. Critical lesson: proxy-scale optimizer results do not always transfer to full scale.

Step 5: Pretrain. Standard decoder-only transformer with RoPE, SwiGLU, RMSNorm. 3B achievable on single 48GB GPU with gradient accumulation. Use spot instances (60–70% cheaper). Checkpoint every 30 minutes. Monitor per-language BPB, not just aggregate loss.

Step 6: Supervised Fine-Tuning. Include all target languages. Native data > translated data. 3K–5K steps sufficient for 3B. Sources: Aya dataset, OASST, language-specific datasets from HuggingFace.

Step 7: Evaluate Comprehensively. At minimum: (1) BPB per language, (2) cross-lingual retrieval, (3) task transfer (train language A, evaluate language B), (4) translation if parallel data included, (5) downstream benchmarks to calibrate expectations.

Decision	Recommendation
Language selection	Prioritize linguistic family over script
Tokenizer	Custom BPE, 32K vocab, balanced training
Data distribution	Over-represent anchor language (30–40%)
Data quality	MinHash dedup + perplexity filter
Recipe validation	20–50 proxy experiments at 100–200M scale
Optimizer	AdamW (β_2=0.95); validate Muon at scale
Schedule	WSD linear-decay, not cosine warm-restarts
Precision	fp16 or bf16
SFT data	All languages, native > translated, 3–5K steps
Translation data	Direct parallel only; EN-mediated insufficient
Compute	Cloud spot instances with checkpointing

Table 10: Recipe summary for building a multilingual foundation model from scratch.

7. Conclusion

We have presented SemiticGPT, a 3B parameter multilingual model covering Hebrew, Arabic, Farsi, and English, trained from scratch on cloud spot instances. Our evaluation across six dimensions reveals both capabilities and limitations, with the most striking finding being massive cross-lingual transfer between Semitic languages for semantic tasks.

Beyond the model itself, we offer a reproducible recipe (Section 6) for building multilingual foundation models for any under-resourced language family. The key principles—linguistically-motivated language

grouping, balanced custom tokenizers, anchor language overrepresentation, proxy-validated training recipes, and comprehensive negative-result reporting—are directly applicable to other language family clusters.

We release the model weights, tokenizer, and training code to support reproducible research.[6]

Future work. (1) Scaling to 7B parameters with improved Arabic SFT data quality; (2) adding more Semitic languages (Amharic, Tigrinya); (3) adapter-based extension to new languages post-pretraining; (4) investigating whether Hebrew→Arabic transfer persists at larger scales.

8. Reproducibility

All code, configurations, and result artifacts are available at https://github.com/fatherRonnen/semitic-gpt. The repository mirrors the paper's pipeline:

Paper Section	Repo Path	Contents
Tokenizer (§3.1)	`tokenizer/`	Config, script, fertility report
Data (§3.2)	`data/`	Collection scripts, manifests
Proxy (§2)	`proxy/`	32 configs, results, rationale
Pretraining (§3.4)	`pretrain/`	Scripts, hyperparams, logs
SFT (§3.5)	`sft/`	6 configs, logs, data recipes
Evaluation (§4)	`eval/`	Scripts, prompts, result JSONs
Tables/Figures	`paper/tables/`	Machine-readable result data

Table 11: Mapping from paper sections to repository artifacts.

Model weights (12.5GB base, 6GB SFT) will be uploaded to HuggingFace. The tokenizer binary (817KB) is available from the same source. Large training data files are on AWS S3; the repository contains collection scripts and metadata sufficient to reconstruct the corpus from public sources.

9. Limitations

- **Single-run results**: All experiments report single-run results without confidence intervals or multiple seeds. Some observed differences (particularly <5 percentage points) may not be statistically significant.
- **Evaluation harness**: Generative classification evaluations produce implausibly low baselines (e.g., 4% on a 4-class task with 25% chance). We believe this reflects prompt/parsing issues rather than anti-correlated model behavior. Relative improvements between configurations remain informative, but absolute numbers should be interpreted cautiously.
- **Small evaluation sets**: 200 samples per language suffices for large effects (5.5%→49%) but not small ones.
- **Scale**: At 3B parameters, the model is below the threshold for complex reasoning. Our findings may not hold at larger scales.
- **Confounded variables**: We cannot fully isolate the effect of linguistic family from pretraining data proportion. Farsi receives 15% vs Hebrew's 40%.
- **No external baselines**: We do not compare against mBERT, XLM-R, or NLLB on the same tasks.
- **Translation quality**: ChrF scores remain low (max 18.7%), making it difficult to distinguish meaningful translation from noise.
- **Generalizability**: Demonstrated on one language cluster only.

[6] Repository: https://github.com/fatherRonnen/semitic-gpt. Tag `v0.1-paper-draft` freezes the exact configs and result files used in this paper.

References

W. Antoun, F. Baly, and H. Hajj. AraGPT2: Pre-trained transformer for Arabic language generation. In *Proc. WANLP*, 2021.

L. Bandarkar et al. The Belebele benchmark: a parallel reading comprehension dataset in 122 language variants. *arXiv:2308.16884*, 2023.

P. Joshi, S. Santy, A. Buber, et al. The state and fate of linguistic diversity and inclusion in the NLP world. In *Proc. ACL*, 2020.

A. Koubâa and A. Ammar. ArabianGPT: Native Arabic GPT-based large language model. *arXiv:2402.15313*, 2024.

Marco-LLM Team. Marco-LLM: Bridging languages via massive multilingual training for cross-lingual enhancement. *arXiv:2412.04003*, 2024.

M. Popović. chrF: character n-gram F-score for automatic MT evaluation. In *Proc. WMT*, 2015.

A. Rivas et al. HeArBERT: A bilingual model for Arabic-Hebrew translation using transliteration. *arXiv:2402.16065*, 2024.

N. Sengupta et al. Jais and Jais-chat: Arabic-centric foundation and instruction-tuned open generative large language models. *arXiv:2308.16149*, 2023.

L.B. Allal et al. SmolLM3: The complete blueprint of a state-of-the-art 3B parameter LLM. HuggingFace Technical Report, 2025.

A. Karpathy. autoresearch: Autonomous AI-driven ML experimentation. GitHub repository, 2026.

R. Slasky. Autonomous AI-driven Hebrew language model optimization: From random search to optimizer-architecture co-discovery. Independent research report, 2026.